Introduction to Docker volumes and bind mounts

In this train, we'll explore Docker volumes and bind mounts, the scenarios in which each is used, and how they differ, providing foundational knowledge for data persistence in Docker.

Learning objectives

- Understand Docker volumes and bind mounts.
- Identify the scenarios in which each is used.
- Recognise the key differences between volumes and bind mounts

Docker Volumes

Docker volumes are used for persistent data storage in containers. They are managed by Docker and stored outside the container's filesystem, meaning the data persists even if the container is removed or recreated.

Key Features of Docker volumes:

- **Managed by Docker**: Docker handles volume creation and management, providing commands to list, inspect, and remove volumes.
- **Persistent Data**: Data stored in volumes is independent of the container lifecycle, meaning it is retained even if the container is deleted.
- **Better Performance**: Volumes are optimized for performance compared to bind mounts, especially on Docker Desktop (Windows and macOS).
- **Shared Across Containers**: Volumes can be shared between multiple containers, enabling data consistency across containers.

Lets imagine we have a MySQL database container (called **my_db**) and we want it to have data persistence. To do this we can create a volume by running the following command:

```
docker volume create my_volume
```

To mount your volume onto a container run the following command:

```
docker run -d --name my_db -v my_volume:/var/lib/mysql mysql
```

Explanation:
Here we mount the my_volume volume to the /var/lib/mysql directory inside the container, where MySQL stores its data.

Bind Mounts

Bind mounts allow you to mount a file or directory from the host system into the container's filesystem. This means the container has direct access to files on the host machine.

Key Features of bing mounts:

- **Not Managed by Docker**: Bind mounts are not managed by Docker, meaning they rely on the host's filesystem.
- **Direct Mapping**: A specific file or directory from the host system is mounted into the container.
- **Ideal for Development**: Bind mounts are often used in development environments because they allow real-time changes to files on the host to be reflected inside the container.
- **No Data Isolation**: Data in bind mounts is directly tied to the host's filesystem, and deleting the container does not affect the data.

To use a bind mount, specify the host file or directory followed by the container's mount point:

```
docker run -d --name my_app -v /path/on/host:/path/in/container my_image
```

Explanation:
Here we mount the /path/to/code directory on the host to the /app directory in the container, allowing for real-time code changes.

Differences Between Volumes and Bind Mounts

While both volumes and bind mounts allow containers to access data from the host filesystem, there are key differences in how they are managed and used:

Feature	Volumes	Bind Mounts
Management	Managed by Docker	Managed by the host system

Feature	Volumes	Bind Mounts
Data Persistence	Data is persisted even if the container is removed	Data is tied directly to the host filesystem
Performance	Optimized for performance, especially on Docker Desktop	Can be less performant, especially on non-Linux systems
Use Cases	Ideal for production environments, databases, and shared data	Ideal for development environments where host changes need to be reflected instantly
Security	More secure, as Docker manages the volume's location	Can expose sensitive host files to the container

When to Use Volumes vs Bind Mounts

When to use volumes:

- **Production Environments**: Volumes are preferred in production because they provide better data management, security, and isolation.
- **Database Storage**: Volumes are commonly used for database storage, ensuring data persists even if containers are stopped or recreated.
- **Data Sharing**: Volumes are ideal when multiple containers need access to the same data.

When to use bind mounts:

- **Development Environments**: Bind mounts are great for development, where you need live updates from the host system to the container.
- **Access to Host Files**: Bind mounts are useful if you need to give the container access to specific files or directories on the host system.
- **Testing and Debugging**: Bind mounts allow you to quickly edit files on the host and have those changes reflected immediately inside the container.

Conclusion

We've explored the concepts of Docker volumes and bind mounts, highlighting the scenarios where each is useful and their key differences.

Key Takeaways ✖

- Volumes are managed by Docker, persist data, and are ideal for production environments.
- Bind mounts allow direct access to host files and are often used in development environments.